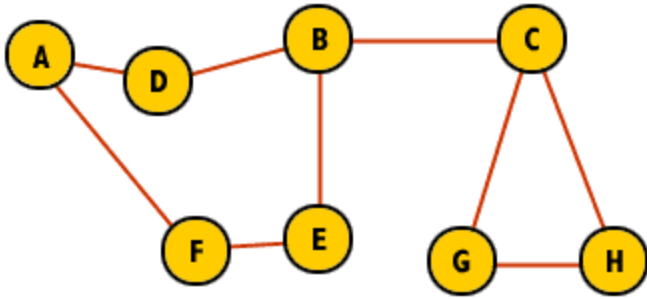


Topic 7.4

7.4: Graphs

Mathematically, a *graph* consists of a set of vertices (*nodes*) and a set of edges (*arcs*). Pictorially, a graph looks like some circles (the nodes) with some lines between them (the arcs):



In set notation, given V is the set of vertices and E the set of edges,

$$V = \{A, B, C, D, E, F, G, H\}$$

$$E = \{(A, D), (A, F), (B, D), (B, E), (B, C), (C, G), (C, H), (E, F), (G, H)\}$$

Two vertices are *adjacent* if there is an edge between them. Thus:

A and D are adjacent.

A and E are *not* adjacent.

The *degree* of a vertex is the number of edges associated with it.

A *walk* is a sequence of nodes such that there is an edge between each pair of nodes in the sequence, such as:

ADBEBCHCB

A *trail* is a walk such that no *edge* appears twice:

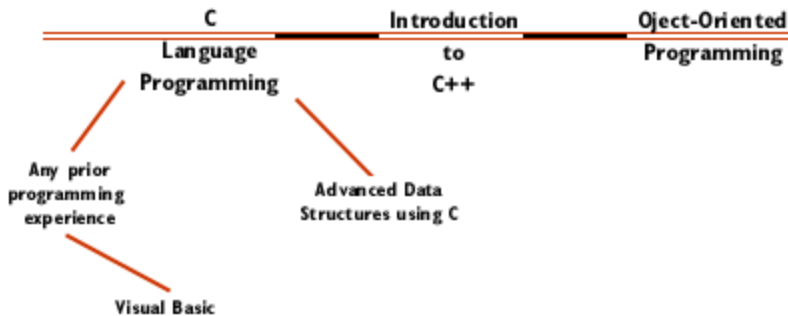
CHGCBDA

A *path* is a trail such that no *vertex* appears twice:

HGCBEFAD

A graph is *connected* if there is a path from each node to every other node. A graph is *acyclic* if it has no cycles. The graph shown above has 2 cycles in it and is *not* acyclic.

An acyclic graph is also known as a *forest*. An acyclic, connected graph is known as a *tree*. In an *undirected* graph, as shown above, the edges have no "direction." In a *directed* graph, the edges have a direction, which conveys important information. For example, a directed graph can be used to show in what order a set of college courses must be taken:



The *indegree* of a node in a directed graph is the number of arcs into the node.

The *outdegree* of a node in a directed graph is the number of arcs out of the node.

Vertices may have information associated with them, such as the name of cities. Edges may have information associated with them, such as distances. These are often referred to as *edge weights*.

Graphs have many applications:

- They serve as PERT charts for identifying the sequence in which tasks must be done.
- They determine the connectivity of networks and how to route traffic.
- They determine the total bandwidth of networks.
- They form the basis for the file system of many operating systems (UNIX, DOS, Mac, VMS, etc.)
- In compilers, they are used to keep track of linear code sequences and branching (gotos).
- They're used to compute distances between cities.

Graphs may be represented as a list of vertices and a list of edges. They may also be represented as a 2-dimensional array A (matrix), such that

$A[i][j]$ is 1 if vertex i is adjacent to vertex j , and 0 otherwise.

Two vertices are adjacent if there is an edge between them. This is called the *adjacency matrix*.

Operations on graphs include:

- creating a graph
- inserting a node
- inserting an edge
- deleting a node
- deleting an edge

- determining if there is a path between two nodes
- determining the shortest path between two nodes

Now, suppose we have a general graph class, and we wish to derive a tree class from it. Note that graphs may have cycles, whereas trees cannot, and graphs don't have to be connected, but trees must be. So, we have more constraints to enforce on a tree than on a graph.

Topic 7.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-7-dot-5>)